EpiScope: A Big Data Surveillance, Forecasting and Response-Planning Platform for Seasonal, Tropical and Epidemic Diseases

Group V^3 | Mohammed Walid, 0222220005101147 (Data Engineer) | Nishat Afroz, 0222220005101145 (Analytics Engineer) | Dodaibul Islam Tuhin, 1903610201784 (Visualisation Engineer)

CSE 4345 Big Data Analytics, Complex Engineering Problem | Department of CSE , Premier University, Semester: 8th

Repository: https://github.com/Walid-encore/episcope (tag v1.0)

Video: https://drive.google.com/file/d/1PjmD2oMMD6eno1HaQXX4A6NV4ZyOpDKX/view?usp=sharing

***Abstract*—*EpiScope is a prototype big-data platform that combines synthetic health-facility encounters, facility stock snapshots, district geography and weather to support disease surveillance and response planning in Bangladesh. A bronze–silver–gold lakehouse on Apache Spark and Parquet ingests incrementally, quarantines defective records with reason codes (37 quality rules) and de-identifies patients before analysis. On top of the gold marts we run EARS C2 and CUSUM outbreak detection, a negative-binomial forecaster with lagged weather, conversion of forecasts into bed and supply demand, a linear programme that allocates scarce supplies with an explicit equity floor, and a seasonal preparedness calendar. The pipeline processes 6.46 million encounters (SF 0.25) in about 12 minutes on a 7.9 GB laptop and 26.1 million (SF 1) in 16.7 minutes on a 16 GB laptop, and exports under 10 MB of suppressed, aggregated JSON to a D3.js dashboard. On the default SF 0.05 data, EARS C2 detected 21 of 34 injected outbreaks at about 0.49 false alerts per district-year, and forecasts cut the weighted interval score to 0.63 of a seasonal-naïve baseline. In a peak-season scenario at SF 0.25, the optimiser lowered severity-weighted unmet demand by 12.1% against population-proportional sharing while guaranteeing every district 25% coverage.*

***Index Terms*—*big data analytics, epidemiological surveillance, Apache Spark, outbreak detection, forecasting, resource allocation, data quality, privacy, Bangladesh.*

I. INTRODUCTION AND PROBLEM ANALYSIS

Bangladesh faces recurring infectious-disease pressure with strong seasonal and geographic structure: dengue follows the monsoon, diarrhoeal disease rises with rainfall and flooding, influenza-like illness (ILI) peaks in the cool season, and malaria is concentrated in hill and border districts. The assignment frames a hypothetical Ministry of Health request to combine health, climate, geography, population and logistics data so that unusual activity is detected early, near-term cases and hospital demand are forecast, scarce supplies are allocated fairly, and seasonal preparedness is planned [1], [10].

The difficulty is not model choice but the data. Inputs differ in schema, update frequency and granularity; records arrive late, are duplicated or are wrong (for example impossible ages); stock snapshots reference unknown facilities; a silent facility followed by a back-filled report can look like an outbreak; and line-level health data must never reach a public dashboard. Large-scale studies of surveillance data show how easily such artefacts mislead [5]. A decision-support system must therefore make provenance, uncertainty, equity and privacy explicit, not only accurate.

EpiScope answers with a reproducible medallion pipeline on Spark, district–disease weekly marts, detection, probabilistic forecasting, demand conversion, constrained allocation and a compact, suppressed serving export. All encounter and stock data are synthetic and must not be read as real surveillance or clinical advice.

II. CEP ATTRIBUTE ANALYSIS AND CONFLICT RESOLUTION

P1 (depth of knowledge) is met by distributed processing, layered storage, epidemiological time-series methods, optimisation and privacy engineering. P2 (conflicting requirements) is handled explicitly through C1–C5 below. P3 (depth of analysis) is shown by threshold, hold-out and allocation-sensitivity experiments instead of a single tuned parameter. P4 (unfamiliar issues) appears as reporting artefacts, late and duplicate records, orphan stock facilities and weather-coverage gaps. P5 (beyond standard codes) is the combination of surveillance algorithms [2], [3], [11] with a project-specific allocation model. P6 (stakeholders) covers national policymakers, epidemiologists, district officers, hospital managers and patients. P7 (interdependence) is the chain from quality rules to detection, forecasting, allocation and the dashboard: a change in one stage changes every later number.

TABLE I. Conflicts and how they were resolved

Conflict	Resolution and evidence
C1 sensitivity vs. false alarms	Threshold swept for EARS C2 and CUSUM against 0.5 false alerts per district-year. Chosen: EARS C2, threshold 7.0, floor 15 cases: 21 of 34 injected outbreaks (62%) at 0.49 false alerts per district-year (Sec. VI-A).
C2 granularity vs. privacy	Counts 1–4 are suppressed in the export and alert rows on suppressed weeks are dropped (60 rows; 13.3% of case cells suppressed at SF 0.25).
C3 efficiency vs. equity	A minimum-coverage floor of 0.25 per district costs 0.4% of weighted unmet demand but lifts the worst district from 0.146 to 0.25 coverage (Sec. VI-C).
C4 freshness vs. cost	Weekly incremental batch instead of streaming: only new files are loaded and only touched epi-weeks are rewritten. Weekly planning does not justify always-on infrastructure.
C5 accuracy vs. interpretability	Transparent detectors and an explicit linear programme; a negative-binomial forecaster with eight weather lags reached a WIS of 0.63 of the seasonal-naïve baseline.

Stakeholder implications follow the same logic: epidemiologists need visible thresholds, district officers need local warnings, hospital managers need demand with an upper quantile, and regulators need suppression. Equity floors and severity weights are policy parameters, not mathematical facts and should be reviewed by domain experts.

III. ARCHITECTURE AND DESIGN DECISIONS

Source files (generated encounters, facility registry, stock snapshots, district boundaries and weather) feed an immutable bronze layer that records source, load time, batch ID and checksum for every file. Spark [6] jobs validate, de-duplicate and de-identify into silver, and aggregate into gold marts: weekly cases by district and disease, cases by reporting lag, reporting completeness, lagged weather, weekly stock, a data-quality report and lineage. Analytics reads only gold and writes alerts, forecasts, demand, allocations and the calendar; a serving exporter turns selected outputs into eleven small JSON files for the D3.js dashboard.

Storage is Parquet on the local file system, partitioned by epi-week, with a Spark SQL catalog (data/catalog) that registers 21 tables across the three layers so that every layer can be queried by name. Spark runs in local mode (PySpark 3.5.9, Python 3.11, Java 17); pandas is used only for small reference data and the final aggregated exports. Orchestration is pipeline/run_all.py, a small DAG runner with per-stage retries, timings and a summary, driven by one make all. A contract stage fails the run if a gold table has missing columns, null or duplicate keys, or invalid district codes.

Alternatives considered. (i) A pandas-only pipeline cannot process millions of rows within student hardware limits and gives no catalog. (ii) A streaming-first design adds operational cost without benefit for a weekly planning cycle (C4). (iii) Delta Lake or Iceberg [7] would give time travel and ACID updates, but plain Parquet with a checksum registry and epi-week rewrites was sufficient and avoids an extra dependency; it is listed as future work. (iv) A single flat export would lose lineage and make late-data corrections hard to reason about.

IV. DATA GENERATION, QUALITY, LINEAGE AND PRIVACY

A. Synthetic data

The seeded, deterministic generator builds 709 facilities, daily district–disease encounter cells for dengue, diarrhoeal disease, ILI and malaria, and facility stock snapshots. It injects 34 outbreaks (dengue 11, diarrhoeal 10, ILI 8, malaria 5) and two reporting artefacts, and writes this ground truth to a separate folder that detection and forecasting never read; it is used only for evaluation. It also injects realistic defects such as duplicate records, late reports, impossible ages and over-long reporting delays. At SF 0.25 the generator produced 6,340,363 base encounters (6,459,739 rows in bronze after defects) and 1,303,735 stock records in 138 s.

B. Quality, quarantine and lineage

Thirty-seven distinct rules cover encounters (16), stock (6), facilities (6), districts (4) and weather (5), well above the required 12: schema and type checks, ranges, referential integrity, duplicate detection, temporal consistency and reporting completeness. Failing rows go to quarantine tables with reason codes; warning-level rules (for example occupied beds above capacity, 0.9% of stock rows in the default run) keep the row and flag it, because overflow is realistic in an outbreak.

TABLE II. Silver results at SF 0.25

Dataset	Bronze rows	Quarantined	Duplicates	Silver rows
Encounters	6,459,739	147,136 (2.3%)	124,486 (1.9%)	6,188,117
Stock	1,303,735	9,083 (0.7%)	12,951 (1.0%)	1,281,701

Stock records whose facility is missing from the registry (6,506 rows, 0.50%, 6,226 distinct identifiers) are excluded from allocation and reported. Gold reconciliation passed with zero failures: the lag-table sums equal weekly cases (67,072 rows checked), and gold cases and reports equal silver (2,643,254 records each). A lineage table with 1,310 records links each gold partition back to bronze batches.

C. Incremental load and the hidden test drop

Bronze registers every file by checksum, so repeating a run loads nothing new. Silver rewrites only the epi-weeks touched by late records. We rehearsed the 14-day drop by generating the next fortnight with the same seed (SF 0.05) and copying it into data/incoming: bronze loaded the 6 new files; silver took in 9,893 rows, quarantined 197, dropped 199 duplicates, replaced 29 existing records and rewrote 6 epi-weeks; the serving data moved to data_as_of 2026-01-12 (8,498 KiB, 8.7 MB); and a second make all reported “nothing to do (idempotent re-run)” in bronze and silver.

The rehearsal exposed one input check that failed when weather ended 12 days before the newest case week. The check now tolerates a trailing weather gap of up to 28 days (forecasts are clipped to the last weather week), while a gap beyond 28 days or weather that starts late still fails.

D. Privacy and data minimisation

Raw patient identifiers exist only in bronze. Patient identifiers are replaced in silver by a salted SHA-256 hash, computed as sha2(salt | trimmed_id, 256). Blank identifiers stay null. The salt is read from the environment variable EPISCOPE_SALT or the untracked .env file; if neither exists, a random 256-bit salt is generated and written to .env. It is never committed, printed or logged, and it is redacted from Spark’s UI and explains output. Exact age (age_years) is validated in silver and rejected if it is missing, non-numeric or above the maximum (rule AGE_INVALID); silver keeps only the patient hash and an age band (0–4, 5–14, 15–24, 25–44, 45–64, 65+), so exact age never leaves bronze. The serving export contains only district–disease–week aggregates: every count from 1 to 4 is replaced by a suppressed flag, and alert rows on suppressed weeks are removed so that an alert cannot reveal a hidden count (6,235 of 47,040 case cells, 13.3%, suppressed at SF

0.25; 60 alert rows dropped; a validator checks that none remain). Suppression follows the small-cell logic behind k-anonymity [9] but does not prove anonymity, and overlapping totals could still allow differencing. Role-based views (PRV-3) are implemented as a demonstration in the dashboard. A district officer sees the chosen district in detail and every other district only as its division's aggregate, and an aggregate is shown only when at least two districts contribute published counts, so it cannot equal one district's count. This limits what the interface shows but is not access control: every exported file can still be downloaded from the browser, so a real deployment would need a server that filters the data before it leaves the secure zone

V. ANALYTICS METHODS

A. Outbreak detection

Detection runs per district and disease on the weekly series in Spark and pandas. EARS C2 [2] scores each week against the mean and standard deviation of a seven-week baseline that skips a guard band; a week alerts when the score is at least 7.0 and the count is at least 15 (a floor that stops tiny counts alerting). CUSUM [11] accumulates positive deviations and alerts above a threshold (22). Both are transparent, unlike the regression of [3], and cheap enough to run on 256 series. Weeks without a usable baseline are marked "not assessed" (1,792 warm-up weeks at SF 0.25) instead of being scored.

Reporting artefacts. The pipeline flags back-fill weeks, where a silent facility reports its backlog at once, and excludes them from alerting. In the artefact tests a back-fill alert disappeared (district 36, dengue: 1 raw alert week, 0 effective), and in a stress test with 50% of facilities silent for three weeks (threshold 7, floor 15) a naive detector raised 31 alert weeks against 9 for the artefact-aware one. At SF 0.25, 10 above-threshold weeks were suppressed as artefacts.

B. Forecasting and demand conversion

A negative-binomial model with up to eight weekly lags of temperature and rainfall (plus seasonality) forecasts one to four weeks ahead per series and returns quantiles; the negative binomial handles over-dispersed counts. A seasonal-naïve forecast is the baseline. Forecasts are evaluated with a rolling-origin backtest (52 origins at SF 0.05; the SF 0.25 run has 55 origins, 49 of them complete for all four horizons) using MAE, 80% interval coverage and the weighted interval score (WIS) [4]. At SF 0.25 the Spark forecast stage took 119.7 s over 66,560 rows and 256 series and wrote 108,544 forecast rows.

Forecast cases become planning quantities (beds, IV saline, NS1 kits, ORS sachets) through disease-specific admission and treatment rates (admission rates observed per district and disease and shrunk toward the national rate with 50 pseudo-cases; a five-day stay for all four diseases, so beds = admissions × 5 ÷ 7; 1.12, 0.36, 0.25 and 0.06 IV saline units per dengue, diarrhoeal, malaria and ILI case; 0.6 NS1 tests per dengue case; and 4 ORS sachets per diarrhoeal case. These are the generator's parameters, not clinical guidance, and a ±25% sensitivity run quantifies their effect). The national planning demand from the latest origin, point and 90th percentile, was: beds 618.6 and 880.3; IV saline 4,829 and 7,067 units; NS1 kits 331.6 and 648.6; ORS 25,035 and 34,920 sachets. The upper quantile is the conservative planning input.

C. Allocation

A linear programme [12] distributes available stock and a national budget across districts to minimise severity-weighted unmet demand. Constraints are demand balance, the budget, optional transport capacity per division and an equity term: a minimum coverage floor per district and a parameter α that trades efficiency for worst-district coverage. District severity weights come from each district's admission and death rates relative to the national rates, shrunk toward the national rate, clipped to 0.5 -- 2.0 and rescaled (0.77 to 1.56 in the SF 0.25 run). Unit costs are relative units, not prices: IV saline 1.0, NS1 kit 3.0 and ORS sachet 0.2. Beds cannot be shipped, so only the three consumables are allocated, and the budget is 50% of the cost of all unmet needs at the 90th-percentile demand. The baseline is allocated in proportion to population. The recommended α is the largest whose efficiency loss against α = 0 stays within 5% at every budget below full cover.

D. Preparedness calendar

For each division and disease we decompose five years of weekly cases into a seasonal profile, test whether a clear season exists (seasonal strength and peak index), and estimate onset, peak and end weeks with earliest and latest ranges. Supplies are pre-positioned four weeks and campaigns started eight weeks before the estimated onset. The output has 32 rows (8 divisions × 4 diseases). For Chattogram, dengue onset is 22 June 2026, peak 13 July and pre-positioning 25 May; diarrhoeal disease onset 4 May, peak 29 June, pre-positioning 6 April; malaria onset 1 June, peak 13 July, pre-positioning 4 May; ILI onset 9 November 2026 with a peak on 25 January 2027.

VI. EVALUATION

The detection and forecast evaluations (Figs. 1–2, Tables III–V) were run on the default SF 0.05 data set. The data-quality, allocation and runtime results (Table II, Section VI-C and Tables VI–VII) come from the SF 0.25 run, and Table VII adds SF 0.5 and SF 1.

A. Detection and conflict C1

Detectors were scored against the 34 injected outbreaks. An outbreak counts as detected if an alert fires in its district and disease within its active window; false alerts are alert episodes outside any outbreak, expressed per district-year. Fig. 1 plots sensitivity against false alerts for both detectors. At the chosen EARS C2 point (threshold 7.0, floor 15), 21 of 34 outbreaks were detected (62%) with about 0.49 false alerts per district-year summed over the four diseases, just inside the 0.5 budget, and median detection delay was 4.5 to 9 days depending on disease. The CUSUM point at t = 22 (circled) detected 11 of 34 (32%) at 0.46. EARS C2 dominates CUSUM at every false-alarm level, and the circled point is the best CUSUM setting that fits the budget, not the shipped detector.

TABLE III. Detection by disease at the chosen operating points

Disease	Outbreaks	EARS C2 detected	False/dist.-yr	Median days
Dengue	11	6 (0.55)	0.171	6.0
Diarrhoeal	10	6 (0.60)	0.174	5.0
ILI	8	5 (0.63)	0.078	9.0
Malaria	5	4 (0.80)	0.068	4.5
All	34	21 (0.62)	0.49	–

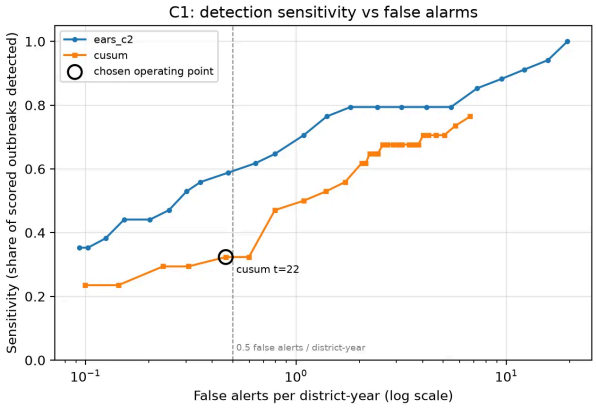


Fig. 1. C1: detection sensitivity versus false alerts per district-year for EARS C2 and CUSUM. Evaluation on the default SF 0.05 data (541,479 cases, 1.3 million encounters)

Out-of-sample check. Because the threshold was tuned on the same outbreaks it is scored on, we also tuned on the early half of the data and tested on the late half, and the reverse (Table IV). EARS C2 kept a sensitivity of 0.60 to 0.65 out of sample (95% intervals about 0.36 to 0.83) against 0.27 to 0.29 for CUSUM, but its false-alert rate was unstable: 0.18 when tested on the early period and 0.80 on the late period, which exceeds the 0.5 budget. The official threshold of 7.0 gives 0.36 false alerts on the early period and 0.63 on the late one. We therefore report the 0.49 figure as an in-sample operating point and expect about 0.2 to 0.8 in practice; the false-alert budget should be re-checked with real data.

TABLE IV. Hold-out detection results

Fold (tuned → tested)	Thr.	Sensitivity [95% CI]	False/dist.-yr	Budget met
EARS early → late	6.5	0.65 [0.41, 0.83]	0.80	no
EARS late → early	8.0	0.60 [0.36, 0.80]	0.18	yes
CUSUM early → late	22	0.29 [0.13, 0.53]	0.55	no
CUSUM late → early	24	0.27 [0.11, 0.52]	0.26	yes

Baseline ablation. The shipped baseline uses seven weeks and leaves 5.4% of weeks unscored. In the ablation a 12-week baseline with a floor of 10 scored 99.7% of weeks and, among the scored outbreaks, raised sensitivity from 0.571 to 0.667 at a slightly lower false-alert rate (0.426 against 0.473), with median delay 5.5 days. We kept the simpler 7-week variant for the delivery run and list the longer baseline as the first improvement.

B. Forecast backtest

TABLE V. Rolling-origin backtest, 1–4 week horizons (SF 0.05 data)

Model	MAE	80% coverage	WIS
Negative binomial + lagged weather	2.420	0.901	1.548
Seasonal naïve	3.563	0.827	2.457

TABLE Va. Backtest by disease (SF 0.05 data).

Disease	MAE model	MAE naïve	Coverage model	Coverage naïve	WIS model	WIS naïve	WIS ratio
Dengue	1.87	2.78	0.94	0.86	1.3	1.95	0.67
Diarrhoeal	3.53	4.69	0.85	0.79	2.26	3.1	0.73
ILI	3.16	5.39	0.89	0.78	1.96	3.85	0.51
Malaria	1.13	1.39	0.93	0.87	0.68	0.93	0.73

The model cuts MAE by 32% and WIS to 0.63 of the baseline. Its 80% intervals cover 90% of outcomes, so they are somewhat wide and conservative, whereas the baseline under-covers (0.83). The model beats the seasonal-naïve baseline on all four diseases (Table Va). The gain is largest for ILI (WIS ratio 0.51) and smallest for diarrhoeal disease (0.73), and 80% interval coverage ranges from 0.85 (diarrhoeal) to 0.94 (dengue). Skill falls only slightly with horizon: the WIS ratio rises from 0.64 at one week to 0.68 at four weeks for dengue, and stays at 0.50 -- 0.52 for ILI, 0.72 -- 0.73 for diarrhoeal disease and 0.73 for malaria.

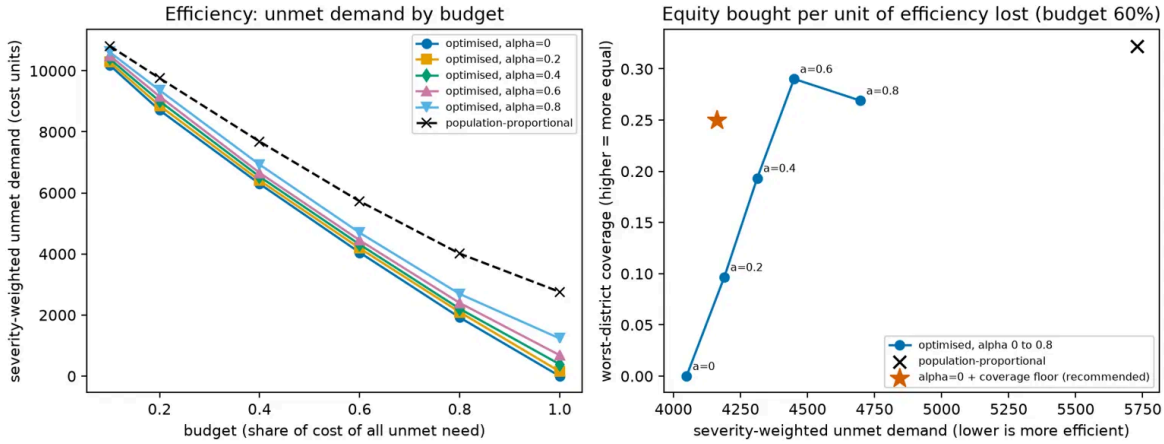


Fig. 2. C3: unmet demand by budget for several equity settings α (left), and equity bought per unit of efficiency lost at a 60% budget (right). SF 0.05 data

C. Allocation: efficiency and equity

Fig. 2 shows that severity-weighted unmet demand falls almost linearly with budget and that every optimised plan beats population-proportional sharing. At a 60% budget, raising α from 0 to 0.8 costs about 16% more unmet demand but lifts worst-district coverage from 0 to about 0.27; the recommended compromise (star) keeps α = 0 and adds a coverage floor, which buys 0.25 worst-district coverage for a much smaller loss in efficiency.

Live plan (SF 0.25, 31 December 2025, 50% budget, floor 0.25). The optimiser leaves a weighted unmet demand of 39.7 against 85.9 for the proportional baseline (54% lower), with minimum district coverage 0.25 against 0.015 and a coverage Gini of 0.23 against 0.52. The baseline spends only 6% of the available coverage because proportional shares cannot be moved to where stock is short, so the peak scenario below is the fairer comparison.

Peak-season scenario (origin 2025-W27, stock as of 6 July 2025). At a budget of 45,906 the optimiser reaches a weighted unmet demand of 43,081 against 48,990 for the baseline (12.1% lower), with total coverage 0.50, minimum district coverage 0.25 and Gini 0.31. The baseline is more even (Gini 0.13, minimum coverage 0.30), so the gain is a real efficiency–equity trade-off, not a free win. Removing the floor lowers unmet demand only slightly further (42,921, 0.4%) but drops the worst district to 0.146 coverage, which is why we recommend the floor.

Sensitivity. The improvement over the baseline stays between 9.4% and 17.0% when any one unit cost is halved or doubled (budget fixed or scaled). Tightening transport capacity to a quarter of its default raises unmet demand by 7.0% and lowers minimum coverage to 0.125, with eight divisions binding. With equal district weights the gain over the baseline shrinks to 0.7% (12.3% with severity weights), so the benefit depends on the severity weighting, a value judgement that policymakers should confirm.

D. Scalability and runtime

The full SF 0.25 run (6,459,739 encounter rows, 1,303,735 stock rows, 140,288 weather rows) completed on a laptop with an Intel Core i3-10100 (4 cores, 8 threads), 7.9 GB of RAM and Windows. Generation took 138 s and the pipeline 710 s (11.8 min), as in Table VI. Table VII adds SF 0.5 (12,970,561 encounter rows, 751 s) and SF 1 (26,076,753 rows, 1,004 s or 16.7 min, inside the 45-minute limit), both on a laptop with an AMD Ryzen 7 7735HS (8 cores, 16 threads) and 16 GB of RAM. On that machine doubling the data from SF 0.5 to SF 1 raised pipeline time by 34%, so the pipeline scales sub-linearly at this size.

TABLE VI. Stage runtimes at SF 0.25 (seconds)

Stage	s	Stage	s	Stage	s
Bronze	77.0	Export	4.4	Allocate	14.1
Silver	191.8	Alerts	22.0	Scenarios	47.0
Gold	136.0	Demand	121.2	Calendar	6.6
Contract	33.6	Catalog	54.8	Serve	1.9
				Total	710.4

Silver, gold and the demand (forecast) stage dominate, which matches where the Spark work is. At SF 0.25 the serving export totalled 9,419 KiB against the 10,000 KiB limit (cases 7,104 KiB, peak allocation 699 KiB, alerts 495 KiB, seasonality 335 KiB, allocation 311 KiB, stock 239 KiB, forecast 214 KiB, calendar 12 KiB, metadata 6 KiB, quality report 4 KiB), measured before the default suppressed flag was dropped from the weekly rows; the default SF 0.05 export is 8,508 KiB (8.7 MB). A size valve trims the oldest weeks if the total would exceed the limit, never below 52 weeks per disease and logging every trim; it did not fire at SF 0.25. The served windows are 262 weeks for dengue and diarrhoeal disease, 159 weeks for ILI (from 22 December 2022, which cuts its first season) and 52 weeks for malaria.

TABLE VII. Scaling runs

SF	Encounter Rows	Pipeline Time	Machine
0.25	6,459,739	710 s	Intel Core i3-10100, 7.9 GB RAM, Windows
0.5	12,970,561	751 s	AMD Ryzen 7 7735HS (8 cores / 16 threads), 16 GB RAM (15.2 usable), Windows
1	26,076,753	1,004 s	AMD Ryzen 7 7735HS (8 cores / 16 threads), 16 GB RAM (15.2 usable), Windows

E. Spark optimisations

We profiled the gold stage at SF 0.5 in the Spark UI (Spark 3.5.9, local mode) on an MSI Thin A15 laptop (AMD Ryzen 7 7735HS, 8 cores/16 threads, 16 GB RAM with 15.2 GB usable, Windows). Reading the encounters was not the bottleneck: gold_lineage scans them four times and takes 2.7–4.4 s. The time went elsewhere. The longest job was a single task that took 29 s to write a 40-row table (Fig. 3a), and each of the four week-grid collections took 9–10 s, with all 16 tasks lasting 10 s and using no memory (Fig. 4). Every one of these steps builds a DataFrame from a Python list. A micro-benchmark (docs/experiments/eng3_pyrdd_test.py) isolates the cost: collecting 262 rows takes 9.5–10.9 s with 16 partitions, 0.73–0.75 s with one, and 0.08–0.37 s with no Python at all. The cost follows the number of partitions, not the number of rows. We attribute it to PySpark starting a Python worker process per partition on Windows, but we did not instrument process creation.

Two changes follow. (1) week_grid now builds the weekly calendar inside Spark (sequence and explode), so collecting it no longer waits for 16 Python workers. (2) The district–disease–week grids and the three reconciliation rows, which still start in Python, are sent to Spark as one partition instead of 16. Neither change alters the results. An exact row-by-row comparison (exceptAll) of all eight gold tables against the unmodified pipeline's output found no differing rows at SF 0.5 and SF 1, the three reconciliation checks passed in every run, and the 17 gold unit tests pass.

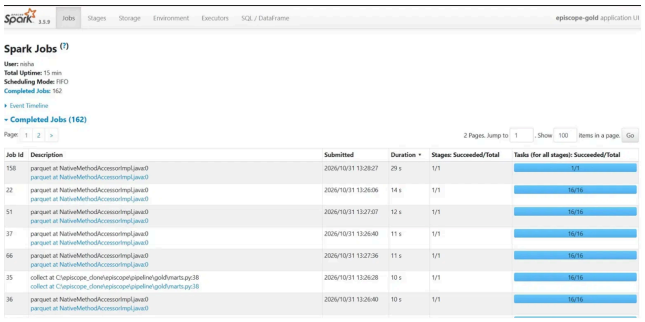
Table VIII lists every run. Identical code varied by up to 28% between runs (the baseline spans 165.0–217.9 s), so we compare means. The slowest run after the changes (92.0 s) is still faster than the fastest baseline run (165.0 s). At SF 0.5 the mean gold time fell from 191.2 s to 83.7 s (−56%). Change 2 alone shortened the table-building phase by 31% and the reconciliation and quality-report phase by 74%. The longest Spark job fell from 29 s to 5 s, and the seven longest jobs from 97 s to 23 s in total (Fig. 3). At SF 1 a single baseline run (207.6 s) is compared with a mean of 72.0 s, so we treat SF 1 as supporting evidence only.

We claim no gain from two other candidates. The broadcast join of the facility registry and adaptive query execution were already enabled in the baseline code. An incremental gold rebuild of only the touched epi-weeks was not implemented, and gold still rebuilds all 262 weeks. The gain removes a start-up cost that we measured only on Windows. Where PySpark can reuse its workers, as on Linux, we expect it to be smaller.

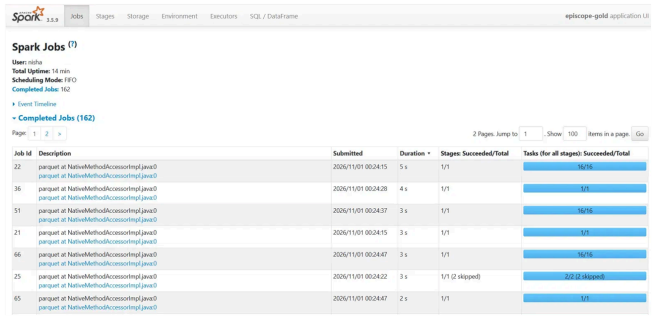
TABLE VIII. Gold-stage wall time (s), mean of the runs listed below, MSI Thin A15 laptop.

Optimisation	Scale	Before (s)	After (s)	Gain
1. Week grid built in Spark	SF 0.5	191.2	141.8	−49.4 s (−25.8%)
2. Python-built grids and reconciliation rows as one partition	SF 0.5	141.8	83.7	−58.1 s (−40.9%)
1 + 2 combined	SF 0.5	191.2	83.7	−107.5 s (−56.2%)
1 + 2 combined	SF 1	207.6*	72	−135.6 s (−65.3%)

Runs (s). SF 0.5 baseline: 165.0, 190.7, 217.9. After change 1: 149.7, 133.9. Final code: 92.0, 79.8, 79.4. SF 1: final code 82.4, 61.6. *Single run of the unmodified pipeline. Times are the "Gold written" times and exclude Spark start-up. Sources: docs/experiments/eng3_*.log and scaling_sf05/sf1_pipeline.log.

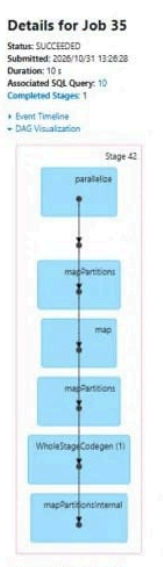


(a) eng3_before_jobs

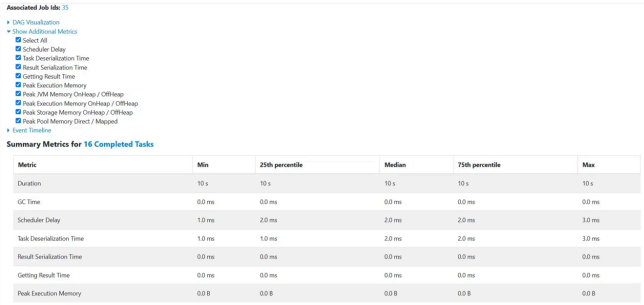


(b) eng3_after_jobs

Fig. 3. Spark UI Jobs tab for the gold stage at SF 0.5, sorted by duration. (a) Baseline run (190.7 s): the longest job is a 29 s single task, and the week-grid collect (job 35) takes 10 s. (b) Final code (79.4 s): the longest job takes 5 s.



(a) eng3_before_job35_dag



(b) eng3_before_stage42

Fig. 4. The week-grid collects before the fix. (a) Job 35: one stage (42) of 16 tasks, built from a Python list. (b) Stage 42 summary metrics: all 16 tasks last 10 s with 0 B peak memory and under 3 ms of scheduling and deserialisation.

VII. DASHBOARD DESIGN RATIONALE AND USABILITY

The dashboard is a static D3.js v7 [8] page that reads only the eleven JSON files in serving/data (8.7 MB for the default SF 0.05 data), never line-level data. Design choices follow from the privacy and size rules: suppressed cells are drawn as a distinct hatched state instead of zero, every chart shows the data-as-of date from freshness.json, the same window is served per disease so views stay aligned, and uncertainty (forecast bands, alert thresholds) is shown beside the observed series so that a forecast is never read as a promise. The five required views are a district choropleth with alert markers, a time series with forecasts and thresholds, a seasonal heatmap, an allocation view comparing demand, stock and recommendation, and a ranked early-warning table, linked by district, disease and time selection.

The dashboard has the five required views and three supporting panels. The choropleth shows weekly incidence per 100,000 with alert markers; the time series shows observed cases, the forecast median with a q10–q90 band, and an alert-score strip against its threshold, with a brush on an overview chart that sets the week range; the seasonal heatmap shows the seasonal index by division and epi-week together with the preparedness calendar; the allocation view compares forecast demand, current stock and the optimised and population-proportional allocations beside an efficiency–equity curve; and the early-warning table ranks districts by severity, then score. A what-if panel (budget, equity floor, detection threshold), a data-quality panel and the role view complete the interface. All views share one state (disease, district, week, week range and the what-if settings): selecting a district on the map, in the table or on an allocation bar selects it everywhere, brushing sets the range, and the sliders update alerts and allocations together.

We used ColorBrewer sequential scales (YlOrRd for incidence, YlGnBu for the seasonal index), viridis for budget lines and the Okabe–Ito palette for allocation bars. Colour never carries meaning alone: suppressed and low-volume cells are hatched, severity uses triangles plus text, and table rows can be reached by keyboard. There is no dedicated mobile view; the layout stacks on narrow screens but was not tested on phones.

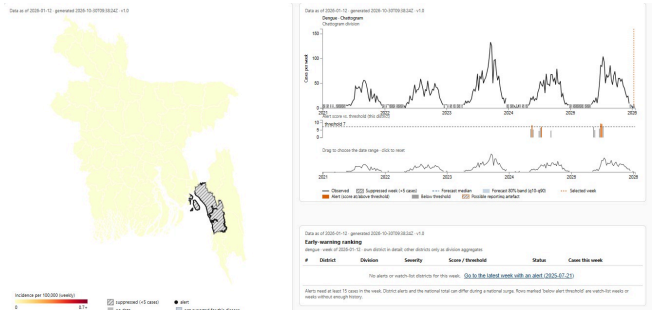


Fig. 5. Selecting a district links the map, time series and ranking.

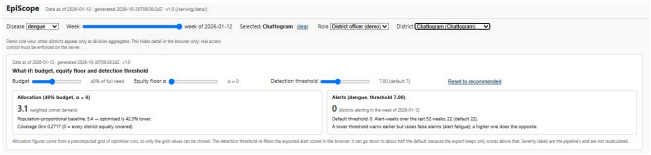


Fig. 6. Budget, equity floor and threshold sliders update allocations and alerts

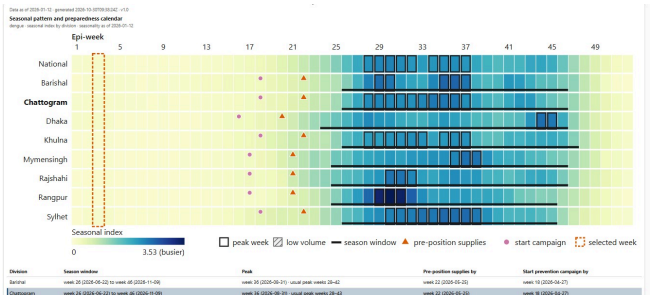


Fig. 7. Seasonal index by division with calendar markers.

VIII. ETHICAL, SOCIETAL AND SUSTAINABILITY CONSIDERATIONS

EpiScope is a teaching prototype on synthetic data. It must not support individual clinical decisions or allocate real supplies without validation, governance and human oversight; local public-health officers keep decision

authority. False negatives delay a response, while false positives divert scarce resources and cause alert fatigue; our own hold-out results (Sec. VI-A) show the false-alert rate can vary several-fold between periods. Ranking districts may stigmatise communities if shown without context, so the dashboard pairs rankings with uncertainty, freshness and method notes.

Privacy rests on de-identification before silver, aggregation and small-count suppression, with bronze access restricted and purpose-limited; suppression must still be checked against differencing across overlapping totals, and differential privacy would be a stronger control for any public release. Equity is the central policy choice: severity weighting reduces total harm but can concentrate supplies, and the coverage floor guards low-volume districts at a measured cost of 0.4% efficiency. For sustainability we process in weekly batches, store compact columnar data, export only aggregates and avoid always-on streaming. Secrets such as the salt are never committed, and AI assistance is disclosed in AI_USAGE.md.

IX. LIMITATIONS AND FUTURE WORK

- **Synthetic data.** All sensitivities and accuracies describe injected outbreaks; real data are noisier. Detection tuning is in-sample, and the out-of-sample false-alert rate varied from 0.18 to 0.80.
- **Detector baseline.** The shipped 7-week baseline leaves 5.4% of weeks unscored; a 12-week baseline scored better in the ablation.
- **Windows.** The ILI window begins inside its first season and malaria is served for 52 weeks only, which limits seasonal validation in the dashboard (the analytics use full history).
- **Allocation.** Stock and demand are national totals split by district; transport and delivery lead times are only a capacity factor; the baseline shows much lower coverage in the live plan, so results depend on the baseline definition.
- **Scale and evidence.** All runs used Spark in local mode on a single laptop (SF 1 in 1,004 s on 16 GB of RAM); behaviour on a cluster is untested.

Future work: longer baselines and a Farrington-style regression detector [3], probabilistic calibration checks, explicit delivery lead times, enforced role-based access control (the current role view is a demonstration only), formal privacy for public releases, time travel with Delta or Iceberg [7], and validation on authorised real surveillance data.

X. CONCLUSION

EpiScope shows an end-to-end, reproducible path from raw encounters to a privacy-aware planning dashboard: a 6.5-million-row run finishes in under 12 minutes on a small laptop, defects are quarantined and reconciled, detection finds about six in ten injected outbreaks at roughly one false alert per two district-years, forecasts beat a seasonal baseline, and the allocation model makes the efficiency–equity trade-off explicit. The main open risks are the unstable false-alert rate out of sample and the need to validate every assumption on real data before operational use.

REFERENCES

[1] CSE 4345 Big Data Analytics, "Complex engineering problem: EpiScope," course assignment brief, Premier University, Chattogram, 2026.

[2] L. Hutwagner, W. Thompson, G. M. Seeman, and T. Treadwell, "The bioterrorism preparedness and response Early Aberration Reporting System (EARS)," J. Urban Health, vol. 80, suppl. 1, pp. i89–i96, 2003.

[3] C. P. Farrington, N. J. Andrews, A. D. Beale, and M. A. Catchpole, "A statistical algorithm for the early detection of outbreaks of infectious disease," J. R. Stat. Soc. A, vol. 159, no. 3, pp. 547–563, 1996.

[4] J. Bracher, E. L. Ray, T. Gneiting, and N. G. Reich, "Evaluating epidemic forecasts in an interval format," PLoS Comput. Biol., vol. 17, no. 2, e1008618, 2021.

[5] D. Lazer, R. Kennedy, G. King, and A. Vespignani, "The parable of Google Flu: Traps in big data analysis," Science, vol. 343, no. 6176, pp. 1203–1205, 2014.

[6] M. Zaharia et al., "Apache Spark: A unified engine for big data processing," Commun. ACM, vol. 59, no. 11, pp. 56–65, 2016.

[7] M. Armbrust et al., "Delta Lake: High-performance ACID table storage over cloud object stores," Proc. VLDB Endow., vol. 13, no. 12, pp. 3411–3424, 2020.

[8] M. Bostock, V. Ogievetsky, and J. Heer, "D3: Data-driven documents," IEEE Trans. Vis. Comput. Graphics, vol. 17, no. 12, pp. 2301–2309, 2011.

[9] L. Sweeney, "k-anonymity: A model for protecting privacy," Int. J. Uncertainty, Fuzziness Knowl.-Based Syst., vol. 10, no. 5, pp. 557–570, 2002.

[10] World Health Organization, Technical Guidelines for Integrated Disease Surveillance and Response in the African Region, 3rd ed. Brazzaville, 2019.

[11] E. S. Page, "Continuous inspection schemes," Biometrika, vol. 41, no. 1/2, pp. 100–115, 1954.

[12] G. B. Dantzig, Linear Programming and Extensions. Princeton, NJ, USA: Princeton Univ. Press, 1963.

APPENDIX A — POLICY BRIEF FOR THE DIRECTOR

EpiScope: earlier warnings and fairer preparedness for dengue, diarrhoea, influenza-like illness and malaria

Where we stand today

EpiScope is a working prototype that joins weekly clinic reports, weather, hospital capacity and stock levels for all 64 districts. It was built and tested on practice data created by computer, not on real patients, so the numbers below show what the method can do, not what is happening in the country today.

In the largest test, about 26 million clinic visit records were processed in roughly 17 minutes on an ordinary laptop. The system flagged disease surges, forecast the coming one to four weeks, converted the forecast into the beds and supplies that would be needed, proposed how to share limited supplies between districts, and produced a calendar of when each disease season is expected to begin in each division.

Our three recommendations

1. **Treat a warning as a prompt to check, not as an announcement.** In our tests the system picked up about 6 in 10 of the surges we planted, usually within about 5 to 9 days of their start, and gave roughly one false alarm for every two districts per year. We recommend that a district team confirms each warning with local clinics before acting. *Expected impact:* surges are seen about a week earlier than a weekly paper review would reveal them, at the cost of a few extra checks. The rate of false alarms changed between periods in our tests (from about one in five to four in five per district-year), so it should be reviewed after the first season.
2. **Move supplies and start campaigns before each season, using the calendar.** The calendar gives each division and disease an expected start and peak. We recommend that supplies are in place about four weeks, and campaigns started about eight weeks, before the expected start. *Expected impact:* for example, in Chattogram dengue is expected to start around 22 June and peak in mid-July, so supplies would be positioned by late May and the campaign begin in late April. The dates come from five years of patterns and should be checked against each year's weather.
3. **Share limited supplies by need, with a guaranteed minimum for every district.** In a simulated peak-season week, sharing by need left about 12% less unmet demand than sharing by population. Setting a minimum of one quarter of needs for every district cost only 0.4% of that gain and raised the coverage of the worst-served district from about 15% to 25%. *Expected impact:* fewer shortages where illness is highest, without leaving any district behind. How much weight to give the sickest districts is a policy choice that you and the technical team should agree.

How to read the dashboard

Use the list at the top to choose a disease and click a district on the map; the map, the line chart, the ranking and the seasonal chart all follow. On the map, darker districts have more cases per 100,000 people and a black dot means a warning this week. The ranking lists the districts with warnings, most serious first. The line chart shows past cases, the forecast and a shaded band: the band is meant to contain the real number about eight times in ten, so when supplies are scarce, plan for its upper edge. The seasonal chart shows how busy each week of the year usually is, and the calendar gives the dates to move supplies and start prevention campaigns. The supplies panel compares what is expected to be needed, what is in stock and what we suggest sending to each district, and its sliders let you test a different budget or a fairer minimum share for every district. Hatched cells on the map and line chart mean the real number was very small and has been hidden on purpose to protect privacy; plain grey means no data. Every panel shows the date of its data.

How much to trust it

Forecasts were closer to what happened than a simple "same as last year" guess: the typical error was about a third smaller. The forecast bands were slightly wider than they needed to be, which is the safer direction. Even so, all results come from practice data. Real clinics report late, skip weeks and change how they record illness, so

accuracy on real data will be lower. The first flu-like season in each district is only partly covered, and no result should replace the judgement of an experienced public-health officer.

Protecting people's data

Patient names and identity numbers are replaced by a scrambled code before any analysis, and the key to the scrambling is kept separately and never stored with the code. The dashboard only receives district totals by week. Any number between 1 and 4 is hidden, and a warning is never shown for a week whose number is hidden. Access to the raw records must stay limited to a few named staff. These safeguards lower the risk but do not remove it, so a privacy review is needed before any release outside the Ministry.

Decision requested

Approve a pilot in two or three districts: run EpiScope alongside the current weekly reports for one season, check each warning, compare the supply suggestions with what was actually needed, and decide after the season whether to extend it.

APPENDIX B — CONTRIBUTION MATRIX

Mark Lead (L), Support (S) or no involvement (–). The shares σ_i must sum to 100% and all three members sign.

Work area	**Mohammed Walid** **Data Engineer**	**Nishat Afroz** **Analytics Engineer**	**Dodaibul Islam Tuhin** **Visualisation Engineer**
Problem analysis, proposal, conflict analysis	L	L	L
Synthetic generator and public-data acquisition	L	–	–
Ingestion, lakehouse, incremental loads	L	S	S
Data quality, lineage and privacy controls	L	S	S
Outbreak detection and evaluation	–	L	–
Forecasting, backtesting, demand conversion	–	L	–
Optimisation model and preparedness calendar	S	L	S
Serving layer and D3.js dashboard	L	–	L
Scalability experiment and Spark optimisations	S	L	S
Report and policy brief	S	S	S
Video production	S	S	S
Agreed overall share σ_i (%)	33.33%	33.33%	33.33%
Commits / lines changed (git shortlog -sn)	40	73	12
Signature and Date	**Mohammed Walid** **01/11/2026**	**Nishat Afroz** **01/11/2026**	**Dodaibul Islam Tuhin** **01/11/2026**

APPENDIX C — INDIVIDUAL REFLECTIONS

C.1 Member 1: Mohammed Walid, 0222220005101147 — Data Engineer

I built the data foundation of EpiScope: the bronze layer that registers every file by checksum so a re-run loads nothing twice, the silver layer that quarantines bad rows with reason codes and replaces patient IDs with a salted SHA-256 hash, the gold marts with lineage and reconciliation checks, the Spark SQL catalog, the one-command run, and the serving export that suppresses small counts and stays under 10 MB. I also ran the scaling tests up to SF 0.25 and wrote the run and hidden-drop instructions. The hardest moment was the hidden-drop rehearsal. The pipeline had passed every normal run, but on a second PC, with the next fortnight of data copied in, it stopped at "weather covers every case week". The weather series ended 12 days before the newest case week. I first thought the new data was wrong; after reading the check and the forecast stage together, I saw that the check was too strict for a trailing gap, while an internal gap really is a problem. With my analytics teammate we allowed a gap of up to 28 days, kept the failure for anything longer, added tests and repeated the rehearsal until it passed and the second run did nothing.

I learned that a pipeline is only reliable once someone else can run it from a clean clone with new data, and that agreeing on interfaces early (the serving contract and the size valve) saved us from many late conflicts. I used an AI assistant for drafts and debugging, but it made mistakes, such as a wrong module name and wrong totals in the first report draft. I learned to check everything against logs and code. Next time I would measure the Spark optimisations earlier, run the scale tests in the first week instead of the last, and add an automatic clean-clone test on every merge.

C.2 Nishat Afroz, 0222220005101145 — Analytics Engineer

I was responsible for the analytics layer (AN-1 to AN-10), the SF 0.5 and SF 1 scaling runs (ENG-2) and two Spark optimisations (ENG-3). Results are on the default dataset (SF 0.05) unless I name a scale. For detection I implemented EARS C2 and a one-sided CUSUM, each in pandas and Spark, with a test that checks if the two agree on small data. The alert stage runs EARS C2 on Spark, and CUSUM stays in pandas for comparison. I chose threshold 7.0 with a 15-case floor: 21 of 34 injected outbreaks detected at 0.49 false alerts per district-year. That point was tuned on the same outbreaks, so I added a temporal hold-out. Out of sample it detects 20 of 32 (sensitivity 0.625) with 0.54 false alerts, slightly above my 0.5 target.

The forecast is a negative-binomial model with weather lagged 4 to 8 weeks. Over a 52-week backtest its MAE is 2.4, against 3.6 for seasonal-naive. I converted forecasts to resource demand, solved the allocation as a linear programme, and built the preparedness calendar and an SEIR scenario module (R0 about 2.0). At the peak-season origin, the optimised plan leaves 22.7% less severity-weighted need unmet than a population-proportional split, but its worst district gets 0% coverage. A 25% coverage floor lifts that district to 25% for about 2.5% more unmet demand, which still leaves a 20.7% gain. With the same floor, the gain at SF 0.5 and SF 1 is 12.2% and 12.4%, so I report it per scale. The Data Engineer ran SF 0.25 and those figures are in the report, as his machine did not support the requirement for running SF 0.5 and SF 1, I did those in my machine along with Spark Optimization. The hardest moment was SF 1. The pipeline took about 17 minutes (23 with data generation, measured before the gold changes), well inside the 45-minute limit, but the server stage failed: the dashboard data was 10.11 MB, over the 10 MB limit (VIS-4). I committed the failing log instead of rerunning quietly, then removed the cause by leaving the default suppressed:false flag out of the weekly rows. Rerunning only serve gave 9.37 MB.

I learned three things from the scale runs. First, alerts rose from 253 at SF 0.05 to 1,184 at SF 0.5 and 1,546 at SF 1, so a threshold tuned on the small dataset needs re-tuning. I did not redo the evaluation at those scales. Second, the optimiser's gain depends on the severity weights: with equal weights it falls below 0.1% at SF 0.5 and SF 1. Third, my first guess about the slow gold stage, repeated scans of silver, was wrong. The Spark UI showed that building a 262-row week table from a Python list took about 10 s, against under 0.4 s when Spark built it, because the cost followed the number of partitions, not the rows. Fixing that, and the same pattern in the grids and reconciliation rows, cut the mean gold time at SF 0.5 from 191 s to 84 s with all eight gold tables unchanged. I learned to measure before optimising. I recorded the serve fix as a change in the serving contract (v0.4), so my Visualisation Engineer's dashboard stayed consistent with the data, and I built my scaling comparison on the Data Engineer's SF 0.25 run.

Next time I would agree on one dataset size and one final full run with my teammates early, make the alert floor relative to a series' size, run all three scale factors on one machine so the timings compare, and test a clean clone sooner.

C.3 Member 3: **Dodaibul Islam Tuhin, 1903610201784 —** ***Visualisation Engineer***

I built the D3.js v7 dashboard (VIS-1 to VIS-6) as a static page that reads only the aggregated exports. It has the five required coordinated views: a district choropleth of incidence with alert markers, a time series with forecast bands, an alert-score strip and a brush for the week range, a seasonal heatmap with the preparedness calendar, an allocation view with the efficiency–equity curve, and a ranked early-warning table. I added a what-if panel for budget, equity floor and detection threshold, a data-quality panel, and a demonstration role view for district officers (PRV-3). All views share one state, so a selection in one view moves the others. Suppressed cells are hatched, forecasts always show their band, and every panel shows its data date. I also wrote a checker that tests the real exports against the contract, the small-count rule and the 10 MB limit, and the Dashboard section of the README.

The hardest decision was the size limit. The first export that fit under 10 MB cut ILI to start in November 2023, which removed the February to May 2023 surge, the best evidence that detection works. Because the dashboard draws what the exports hold, I argued with the Data Engineer for freeing space elsewhere: a shorter stock history that my views did not need, fewer equity-floor values in the peak scenario, and a size valve that reacts only to the total. That forced me to fix my allocation view, which assumed every scenario shared one grid; with all-zero rows omitted, a missing row would have read as "nothing allocated". The agreed export keeps the surge. I learned that a contract only protects you if something checks it, and that bugs come from the real shape of the data. My map was one grey block until I found that the boundary file's ring direction is the opposite of what d3-geo expects, and my checker crashed on a null severity that no sample had. I also learned that hiding detail in a browser is presentation, not security. I used an AI assistant for most of the code, but it could not open a browser, so I had to test each view myself.

Next time I would ask for real exports earlier instead of building for long on hand-made samples, run the usability sessions as soon as the first views worked, and add an automated browser test. I did not build a mobile view (VIS-7) and would do that with more time.